

# System Architecture: File-by-File Walkthrough

This document maps the system architecture to the actual source files, explaining the data flow and purpose of each module in sequential order.

---

## ■ Table of Contents

1. [Core Models & Configuration](#1-core-models--configuration)
2. [Data Ingestion & Preprocessing](#2-data-ingestion--preprocessing)
3. [Document Analysis](#3-document-analysis)
4. [Knowledge Graph Construction](#4-knowledge-graph-construction)
5. [Relationship Analysis](#5-relationship-analysis)
6. [Contradiction Detection](#6-contradiction-detection)
7. [Research Gap Discovery](#7-research-gap-discovery)
8. [Explanation & Reporting](#8-explanation--reporting)
9. [Orchestration & CLI](#9-orchestration--cli)

---

## 1. Core Models & Configuration

### [models.py](src/rpra/models.py) — Domain Models

**Purpose:** Defines all data structures used throughout the pipeline.

#### Key Classes:

- **Entity** — represents extracted concepts (objective, methodology, dataset, model, metric, result, limitation, gap)
- **Relation** — represents connections between entities (uses, evaluates-on, outperforms, contradicts, extends, cites, addresses, identifies-gap-in)
- **EvidenceTrail** — structured record of source (doc\_id, section, page, sentence span) for every finding
- **KnowledgeGraph** — NetworkX multigraph representation with JSON serialization
- **RelationshipScore** — composite weighted score across 5 dimensions
- **Contradiction** — detected claim conflicts with confidence levels
- **ResearchGap** — identified unexplored areas with novelty scores

**Usage:** All other modules import and use these classes as their primary data currency.

---

### [config.py](src/rpra/config.py) — Configuration Management

**Purpose:** Loads and validates configuration from config.yaml using Pydantic.

**Key Functions:**

- `load_config()` — reads YAML, validates all required fields
- `validate_weights()` — ensures scoring weights sum to 1.0
- `validate_thresholds()` — checks NLI, gap discovery, and other numeric thresholds

**Validation Rules:**

- LLM provider must be one of: openai, ollama, anthropic
- Weights must sum to 1.0 (objective + methodology + dataset + results + citation)
- Confidence thresholds must be between 0.0–1.0
- File paths must exist

**Used by:** Every downstream module reads config to initialize models and parameters.

---

## 2. Data Ingestion & Preprocessing

### `[ingestion.py](src/rpra/ingestion.py)` — PDF Processing

**Purpose:** Extracts text from PDF files and segments into logical sections.

**Key Functions:**

- `load_pdf(path)` — reads PDF using PyMuPDF, extracts text and page numbers
- `segment_document(text)` — splits text into: abstract, introduction, methodology, results, conclusion, references
- `create_ingested_document()` — returns structured `IngestionDocument` with metadata (filename, total pages, segment boundaries)

**Output:** For each PDF:

```
IngestionDocument {
  filename: "paper.pdf",
  text: "...",
  segments: {
    "abstract": {"text": "...", "start_page": 1},
    "methodology": {"text": "...", "start_page": 3},
    ...
  }
}
```

**Used by:** `classification.py` and `extraction.py` consume ingested documents.

---

## 3. Document Analysis

### `[classification.py](src/rpra/classification.py)` — Document Type Classifier

**Purpose:** Assigns each paper to one of three types: Experimental, Survey/Review, or Methodological.

**Key Functions:**

- `classify_document(ingested_doc)` — analyzes text features (e.g., methodology section length, presence of "survey", experimental keywords)

- Returns: DocumentClassification with type + confidence score

#### Logic Example:

- If methodology section is long + quantitative results present → **Experimental**
- If "survey", "review", "overview" keywords present → **Survey/Review**
- If focus on algorithms/techniques → **Methodological**

**Confidence:** 0.0–1.0 score; < 0.5 triggers manual review flag in report.

#### Used by:

- `extraction.py` uses classification type to prompt-engineer LLM differently per document type
- `reporter.py` includes classification in output

---

## [`extraction.py`](`src/rpra/extraction.py`) — Structured Entity & Relation Extraction

**Purpose:** Uses LLM (GPT-4o-mini) to extract structured entities and relations from ingested documents.

#### Key Functions:

- `extract_entities(ingested_doc, classification)` — calls LLM with constrained JSON schema for entities
- Entity types: objective, methodology, dataset, model, evaluation\_metric, quantitative\_result, limitation, research\_gap
- LLM returns structured JSON; non-conforming outputs discarded and retried (max 2 retries)
- `extract_relations(ingested_doc, entities)` — calls LLM to link extracted entities
- Relation types: uses, evaluates-on, outperforms, contradicts, extends, cites, addresses, identifies-gap-in
- Every relation includes EvidenceTrail (doc\_id, section, page, sentence span)

#### Hallucination Mitigation:

- Constrained JSON schema validation
- Retry logic (up to 2 retries on non-conforming output)
- Audit log of all discarded/retried outputs
- Low LLM temperature (0.0 per config) for deterministic behavior

**Output:** Per document:

```
ExtractionResult {
  doc_id: "paper_1",
  entities: [Entity, Entity, ...],
  relations: [Relation, Relation, ...],
  audit_log: [DiscardedOutput, DiscardedOutput, ...]
}
```

**Used by:** `knowledge_graph.py` receives extraction results to build the KG.

---

## 4. Knowledge Graph Construction

### [`knowledge_graph.py`](`src/rpra/knowledge_graph.py`) — KG Builder & Query Engine

**Purpose:** Constructs and maintains a queryable NetworkX directed multigraph of all extracted knowledge.

**Key Functions:**

- `build_kg(extraction_results)` — creates graph from entities + relations
- Each entity becomes a node with metadata (type, text, doc\_id)
- Each relation becomes an edge with `EvidenceTrail`
- Bridge Entities (shared across  $\geq 2$  documents) promoted to hub nodes
- `query_entities(entity_type, doc_id)` — retrieve entities by type/document
- `query_relations(source_node, relation_type)` — retrieve edges from a node
- `multi_hop_paths(start_node, max_depth=3)` — find connected entities up to depth 3
- `to_json()` / `from_json()` — serialization for persistence and import into graph tools

**Bridge Entity Logic:**

- Scans all entity nodes
- If an entity appears in  $\geq 2$  documents, elevates it to Bridge Entity
- Bridge Entities serve as reasoning hubs for cross-paper connections
- Example: dataset "ImageNet" appears in papers A and B  $\rightarrow$  Bridge Entity enabling  $A \rightarrow \text{dataset} \rightarrow B$  reasoning path

**Used by:**

- `scoring.py` queries the KG for entity/relation overlap
- `contradiction.py` uses the KG to find candidate pairs
- `gap_discovery.py` analyzes KG connectivity for gaps
- `explainer.py` extracts Evidence Graphs from KG subgraphs
- `reporter.py` exports full KG as JSON

---

## 5. Relationship Analysis

### [`scoring.py`](src/rpra/scoring.py) — Weighted Relationship Scoring

**Purpose:** Computes a composite score for each document pair, measuring how related they are.

**Key Functions:**

- `compute_relationship_score(doc_1, doc_2, kg)` — calculates weighted sum:

'''

Score =  $0.25 \cdot S_{\text{obj}} + 0.25 \cdot S_{\text{meth}} + 0.20 \cdot S_{\text{ds}} + 0.20 \cdot S_{\text{res}} + 0.10 \cdot S_{\text{cite}}$

'''

- `similarity_objective(doc_1, doc_2)` — cosine similarity of embedded objectives
- `similarity_methodology(doc_1, doc_2)` — cosine similarity of embedded methodologies
- `similarity_dataset(doc_1, doc_2)` — Jaccard similarity of dataset entity sets
- `similarity_results(doc_1, doc_2)` — Jaccard similarity of quantitative result entities
- `similarity_citation(doc_1, doc_2)` — Jaccard similarity of cited entities

**Embedding:** Uses sentence-transformers/all-MiniLM-L6-v2 for objective/methodology vectors.

**Output:** Per document pair:

```
RelationshipScore {
  doc_1_id: "paper_1",
  doc_2_id: "paper_2",
  score: 0.47,
  component_scores: {
    "objective": 0.32,
    "methodology": 0.51,
    "dataset": 0.40,
    "results_metrics": 0.55,
    "citation": 0.20
  },
  evidence_trail: EvidenceTrail
}
```

**Used by:**

- `contradiction.py` filters candidate pairs by score threshold ( $> 0.4$ )
- `gap_discovery.py` identifies weak connections ( $\text{score} < 0.2$ )
- `reporter.py` ranks and reports top-N related document pairs

---

## 6. Contradiction Detection

### [`contradiction.py`](src/rpra/contradiction.py) — NLI-Based Contradiction Finder

**Purpose:** Detects conflicting claims across papers using multi-signal validation.

**Key Functions:**

- `find_contradictions(kg, scores)` — generates candidate pairs and validates them
- `extract_claim_pairs(doc_1, doc_2, kg)` — finds entities in both papers that could conflict
- `nli_classify(claim_1, claim_2, nli_model)` — uses DeBERTa NLI classifier
- Returns: entailment | contradiction | neutral
- `validate_contradiction(pair, kg, scores)` — applies multi-signal confirmation:

---

Confirmed = NLI(contradiction)

AND (dataset\_compatible OR metric\_comparable)

AND methodology\_similarity  $\geq 0.30$

---

**Candidate Filtering:**

- Includes pairs that share a Bridge Entity (high semantic connection)
- Includes pairs with RelationshipScore  $> 0.4$
- Skips low-similarity pairs to reduce false positives

**Output:** Per contradiction:

```
Contradiction {
  claim_1: (doc_a, entity_x, sentence),
  claim_2: (doc_b, entity_y, sentence),
  nli_label: "contradiction",
  nli_confidence: 0.89,
  signal_fusion_score: 0.72,
  confirmed: True, # or False if only partial signals match
}
```

```
evidence_trail: EvidenceTrail
}
```

**Used by:** `reporter.py` includes all contradictions (confirmed + unconfirmed with confidence) in output.

---

## 7. Research Gap Discovery

### [`gap_discovery.py`](`src/rpra/gap_discovery.py`) — Novelty-Scored Gap Finder

**Purpose:** Identifies unexplored research areas by analyzing weak KG connections.

#### Key Functions:

- `discover_gaps(kg, scores, config)` — scans for candidate gaps
- `identify_weak_connections(kg, scores)` — finds document pairs with:
  - `RelationshipScore < 0.2` (weak connection threshold, configurable)
  - Connected via Bridge Entity (same dataset/model used but different contexts)
- `validate_against_existing_gaps()` — filters out gaps already labeled in KG (limitation/research\_gap entities)
- `compute_novelty_score(entity, kg)` — calculates how unexplored a potential gap is:
  - Inverse of connection density (fewer edges = higher novelty)
  - Penalizes if gap already documented by another paper
- `rank_by_novelty()` — sorts gaps by novelty score

#### Logic Example:

```
Paper A: "We use ImageNet for image classification [novelty: addresses ImageNet classification]"
Paper B: "We use ImageNet for image segmentation [novelty: addresses ImageNet segmentation]"
Bridge Entity: ImageNet
Weak Connection: Both papers use ImageNet but no documented relation between A→B
Discovered Gap: "ImageNet performance gap between classification and segmentation tasks"
Novelty: High (few papers connect these tasks)
```

#### Output: Per gap:

```
ResearchGap {
  entity: Bridge Entity ("ImageNet"),
  doc_pair: (paper_a, paper_b),
  gap_description: "...",
  novelty_score: 0.81,
  connection_density: 0.12,
  evidence_trail: EvidenceTrail
}
```

**Used by:** `reporter.py` includes top-N gaps ranked by novelty score.

---

## 8. Explanation & Reporting

### [`explainer.py`](`src/rpra/explainer.py`) — RAG-Grounded Explanation Generator

**Purpose:** Generates human-readable explanations grounded exclusively in the Evidence Graph.

#### Key Functions:

- `generate_explanation(finding, kg, evidence_subgraph)` — calls LLM with:
- Finding to explain (contradiction, gap, or relationship)
- Pruned Evidence Graph (only supporting edges, max 10 edges per explanation)
- Strict instruction: "Explain using ONLY the provided evidence graph. Do not generate information outside it."
- `extract_evidence_subgraph(finding, kg)` — prunes KG to include only edges supporting the finding
- `validate_explanation(explanation, evidence_subgraph)` — flags if explanation lacks  $\geq 2$  supporting edges (low-confidence)

**Output:** Per finding:

```
Explanation {
  text: "Paper A contradicts Paper B because ...[inline citations to evidence]...",
  evidence_edges: [Edge, Edge, Edge, ...],
  supporting_edge_count: 5,
  confidence: "high", # or "low" if < 2 edges
  grounding_success: True
}
```

**Used by:** `reporter.py` includes explanations in both JSON and Markdown outputs.

---

## **[reporter.py](src/rpra/reporter.py) — Multi-Format Export**

**Purpose:** Exports all findings in JSON (machine-readable) and Markdown (human-readable) formats.

**Key Functions:**

- `generate_json_report(findings, kg, explanations)` — structured JSON export

```
```json
{
  "metadata": {...},
  "contradictions": [...],
  "research_gaps": [...],
  "top_relationships": [...],
  "knowledge_graph": {...}
}
```
```

- `generate_markdown_report(findings, kg, explanations)` — human-readable export with:
- Inline citations: [DocID, Section, Page]
- Formatted tables for contradictions, gaps, related papers
- Executive summary section
- `export_knowledge_graph(kg)` — exports full KG as JSON for import into Gephi/Cytoscape

**Output Files:**

- `report.json` — machine-consumable findings + KG snapshot
- `report.md` — human-readable report with citations
- `knowledge_graph.json` — graph visualization import format

**Used by:** `pipeline.py` calls `reporter` at the end to write output files.

---

## [progress.py](src/rpra/progress.py) — Live Progress & Logging

**Purpose:** Provides real-time CLI feedback and structured JSON pipeline log.

### Key Functions:

- `initialize_progress_panel()` — creates Rich progress display
- `update_progress(stage, status)` — updates live panel (PDF ingestion: 10/50 documents...)
- `log_event(event_type, data)` — records pipeline event to JSON log with timestamp

```
```json
{
  "timestamp": "2026-09-02T14:30:45Z",
  "stage": "extraction",
  "document": "paper_1.pdf",
  "event": "entities_extracted",
  "count": 42,
  "duration_ms": 3200
}
```

### Output:

- Live terminal panel (real-time feedback)
- `pipeline.log` — JSON log for post-hoc analysis

**Used by:** `pipeline.py` integrates progress tracking throughout all stages.

---

## 9. Orchestration & CLI

### [pipeline.py](src/rpra/pipeline.py) — End-to-End Orchestrator

**Purpose:** Orchestrates the entire workflow: ingest → classify → extract → score → detect → discover → explain → report.

### Key Functions:

- `run_pipeline(config)` — master orchestration function:
  1. Load config via `config.py`
  2. Ingest all PDFs via `ingestion.py`
  3. Classify each document via `classification.py`
  4. Extract entities/relations via `extraction.py`
  5. Build KG via `knowledge_graph.py`
  6. Score all document pairs via `scoring.py`



7. Detect contradictions via `contradiction.py`
  8. Discover gaps via `gap_discovery.py`
  9. Generate explanations via `explainer.py`
  10. Export findings via `reporter.py`
- Each step updates progress via `progress.py`
  - Error handling + retry logic for LLM calls

**Incremental Mode:** (future enhancement)

- Track last-processed document set
- Only reprocess new/modified PDFs
- Merge new findings into persisted KG

**Used by:** `cli.py` calls `pipeline.run_pipeline()` on user command.

---

## [cli.py](src/rpra/cli.py) — Command-Line Interface

**Purpose:** Exposes pipeline functionality via CLI commands.

**Commands:**

- `rpra run` — full pipeline execution
- `rpra run --skip-extraction` — demo mode (no LLM API key needed)
- `rpra run --config my-config.yaml` — custom configuration
- `rpra validate-config` — validate configuration without running pipeline

**Entry Point:** Installed as `rpra` command via `pyproject.toml` `entry_points`.

**Used by:** End users run CLI commands to trigger the pipeline.

---

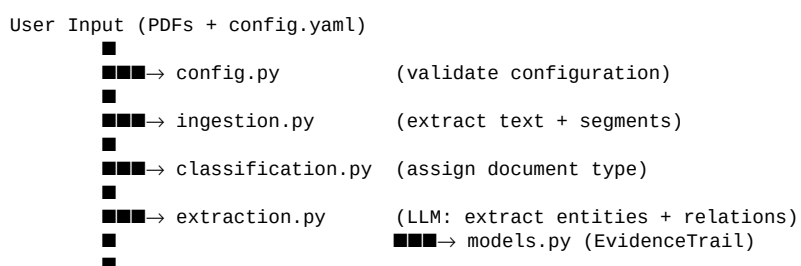
## [\_\_init\_\_.py](src/rpra/\_\_init\_\_.py) — Package Initialization

**Purpose:** Exposes public API for package installation.

**Exports:** Top-level classes and functions for programmatic use.

---

## ■ Complete Data Flow Diagram



```

■■■→ knowledge_graph.py (build KG + Bridge Entities)
■■■→ models.py (KnowledgeGraph)
■■■→ scoring.py (compute relationship scores)
■■■→ models.py (RelationshipScore)
■■■→ contradiction.py (NLI-based detection)
■■■→ models.py (Contradiction)
■■■→ gap_discovery.py (find weak connections + novelty)
■■■→ models.py (ResearchGap)
■■■→ explainer.py (generate grounded explanations)
■■■→ models.py (Explanation)
■■■→ reporter.py (JSON + Markdown export)
■■■→ progress.py (live panel + JSON log)
■■■→ Output Files (report.json, report.md, knowledge_graph.json, pipeline.log)

```

---

## ■ Configuration Dependency Map

```

config.yaml (single source of truth)
■■■→ config.py (validates + loads)
■■■→ extraction.py (LLM provider, model, temperature, max_retries)
■■■→ scoring.py (relationship weights, embedding model)
■■■→ contradiction.py (NLI model, confidence thresholds)
■■■→ gap_discovery.py (weak_connection_threshold)
■■■→ pipeline.py (orchestration parameters)

```

---

## ■ Key Design Principles

| Principle                           | Implementation                                                                                |
|-------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>End-to-end explainability</b>    | Every finding carries EvidenceTrail (doc, section, page, sentence)                            |
| <b>LLM hallucination prevention</b> | Structured schemas, retry logic, grounding in Evidence Graph                                  |
| <b>Configurable pipeline</b>        | All parameters in single config.yaml, validated at startup                                    |
| <b>Bridge entity reasoning</b>      | Cross-document entities enable multi-hop reasoning up to depth 3                              |
| <b>Multi-signal validation</b>      | Contradictions confirmed only via NLI + dataset/metric compatibility + methodology similarity |
| <b>Novelty scoring</b>              | Research gaps ranked by connection density (rarity in KG)                                     |
| <b>Incremental processing</b>       | Architecture designed for only-new-docs reprocessing (future)                                 |

---

## ■ Test Coverage

| Module            | Test File              | Coverage                                                   |
|-------------------|------------------------|------------------------------------------------------------|
| models.py         | test_models.py         | Entity round-trip, KG round-trip, queries, bridge entities |
| config.py         | test_config.py         | Loading, missing keys, invalid weights, file-not-found     |
| classification.py | test_classification.py | Type detection, low-confidence flags                       |
| scoring.py        | test_scoring.py        | Jaccard, cosine, composite score formula                   |

---

\*Last Updated: 2026-09-02 | Architecture Version: 1.0\*